



# Why You Should Choose XiangShan

- Industry-competitive, high-performance, fully-verified and open-source processor
- First-tier performance in RISC-V
- Industrial-grade verification flow including UT, IT, ST + FPGA
- The **only** open-source implementation of RISC-V RVA23 profile
  - Your ideal choice for **Vector** and **Hypervisor** research

# XiangShan Microarchitecture Design Philosophy

---

The XiangShan Team

# XiangShan History Lesson

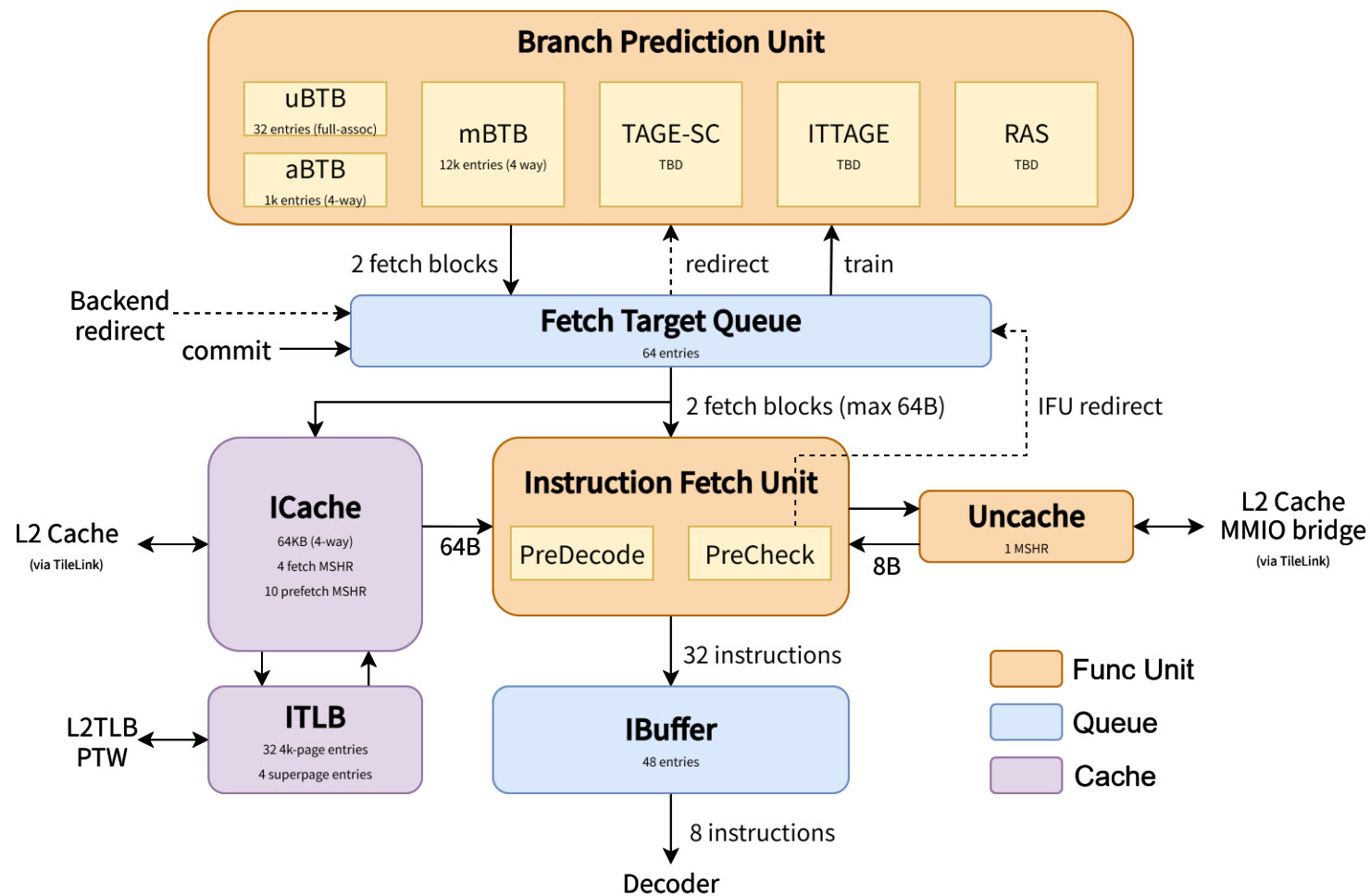
- 1st generation: 雁栖湖 (Yanqihu)
  - 2020/6: first commit of RTL design
  - 2021/7: 28nm tape-out, 1.3GHz
  - SPEC CPU2006 7.01@1GHz, DDR4-1600
- 2nd generation: 南湖 (Nanhu)
  - 2021/5: performance exploration and RTL design
  - 2023/4: GDSII delivery
  - 14-nm tape-out
  - SPEC CPU2006 20@2GHz

# XiangShan History Lesson

- 3rd generation: 昆明湖 (Kunminghu)
  - ISA feature: Vector (V) extension, Hypervisor (H) extension
  - Support RVA23 profile and L2 Cache with CHI protocol
  - Estimated SPEC CPU2006 45@3GHz
- Future: Kunminghu V3
  - Targeted at SPEC 2006 22/GHz
  - Extended multi core support



# Frontend Architecture



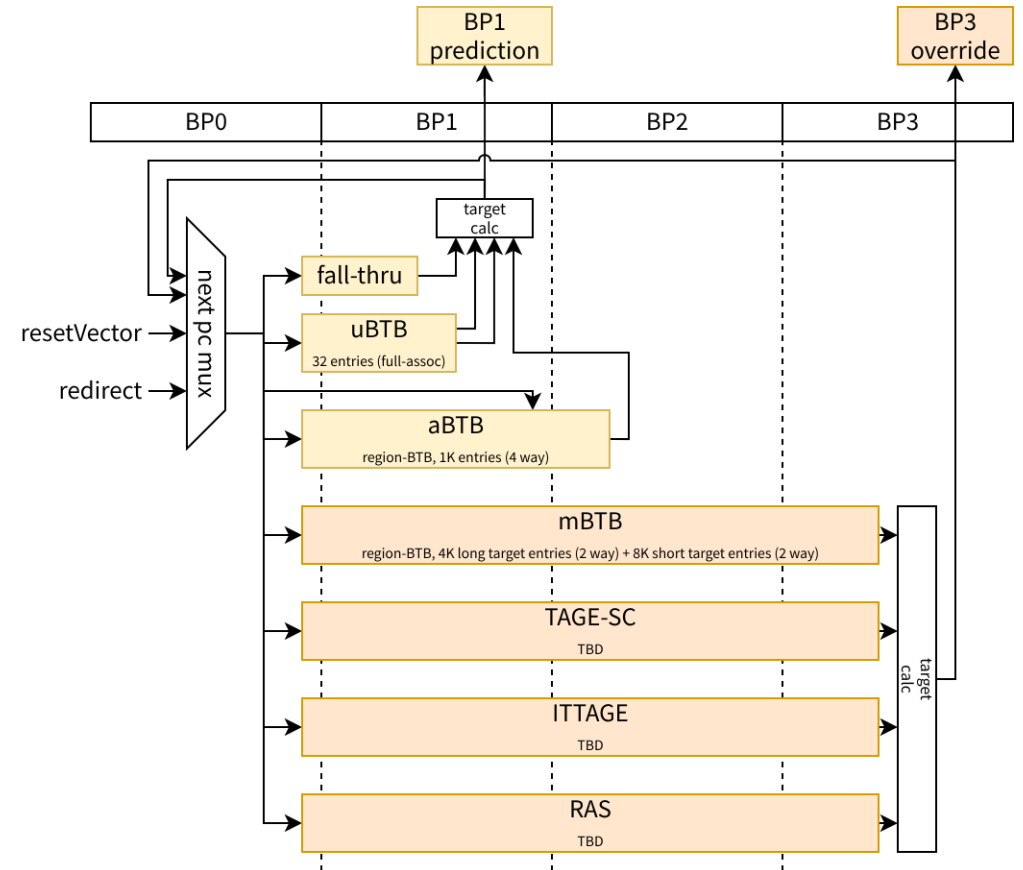


# Frontend Design Philosophy

- Primary goal: provide enough instructions for 8-wide backend
- We need high accuracy BPU and high bandwidth IFU
- Decoupled frontend
- 64B fetch block, maximum 32 instructions
- BTB structure, maximum 8 branches in 64B
  - 4 branches in every aligned 32B
- Limited 2-taken BPU and 2-fetch IFU

# BPU Architecture Overview

- 3-stage prediction pipeline
  - BP1: uBTB + aBTB
  - BP3: mBTB + RAS + TAGE + IT-Tage
- Maximum 8 branches in 64B
- Limited 2-taken support
- No more cross-page predictions
- Half-aligned index





# Limited 2-Taken

- Gain 50% performance of 2-taken with 10% extra area
- Only uBTB will generate 2-taken predictions under some certain conditions
- First branch: static target
  - Same target indirect jump
  - Direct jump
  - Branch
- Second branch: no conflict
  - Branch: TAGE-SC read conflict
  - Indirect: ITTAGE read conflict
  - Consecutive call: RAS read conflict

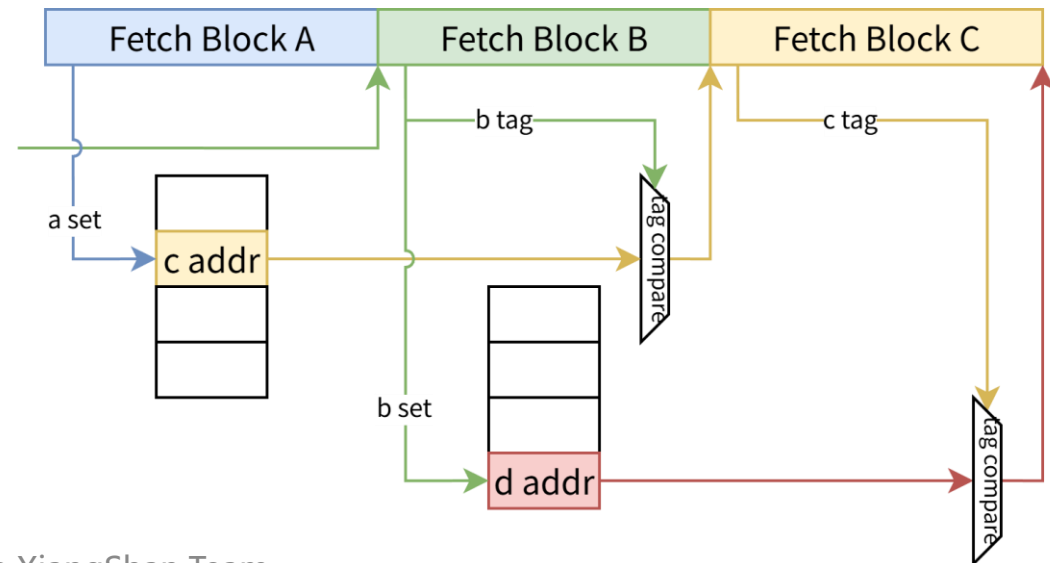
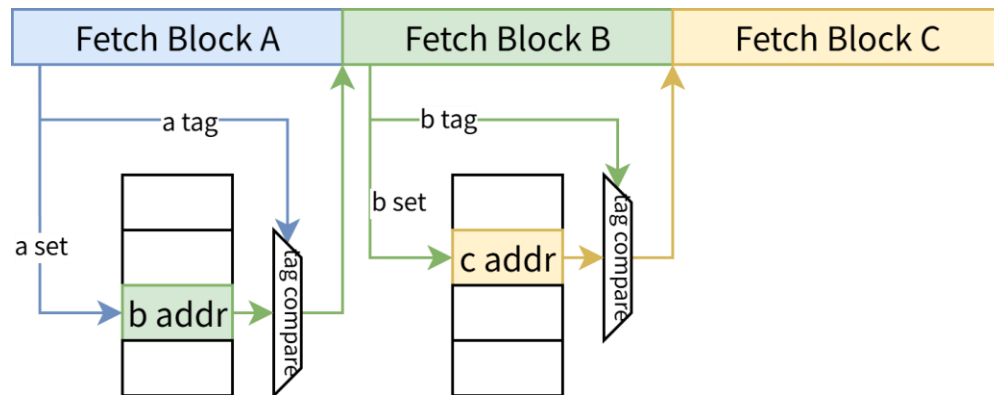
| BR1 \ BR2                    | Branch | Call | Other direct jumps | Ret | Other static indirect jumps | Other dynamic indirect jumps |
|------------------------------|--------|------|--------------------|-----|-----------------------------|------------------------------|
| Branch                       |        |      |                    |     |                             |                              |
| Stable call                  |        |      |                    |     |                             |                              |
| Dynamic call                 |        |      |                    |     |                             |                              |
| Other direct jumps           |        |      |                    |     |                             |                              |
| Ret                          |        |      |                    |     |                             |                              |
| Other static indirect jumps  |        |      |                    |     |                             |                              |
| Other dynamic indirect jumps |        |      |                    |     |                             |                              |



# Micro-BTB and Ahead-BTB

- Large capacity S1 predictor with ahead pipeline technology
- 1K entries (4-way)

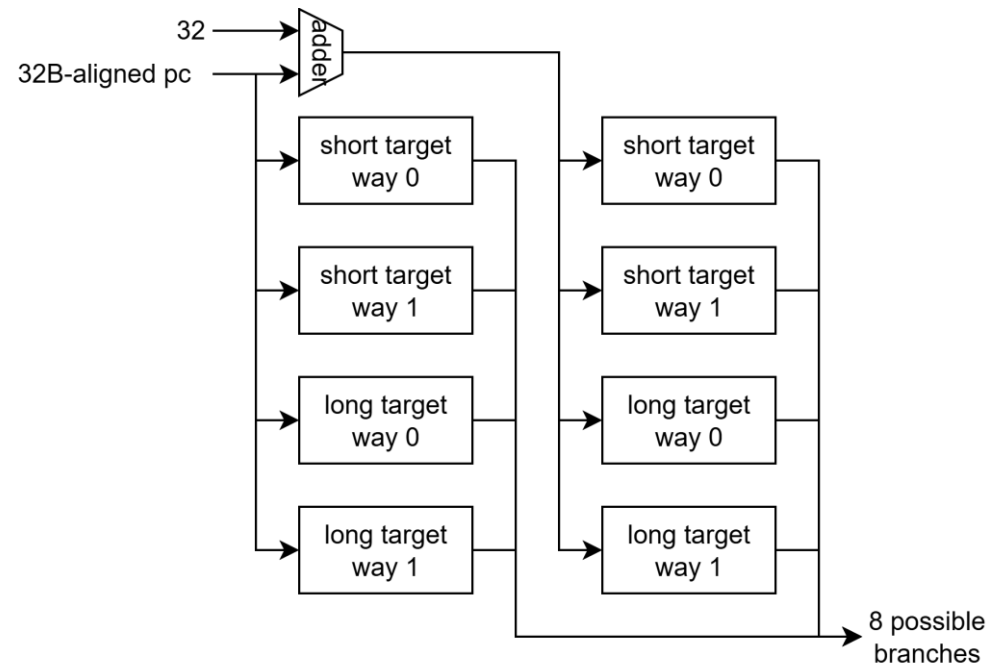
| Predictor    | Index | Tag | Prediction |
|--------------|-------|-----|------------|
| uBTB (left)  | A     | A   | B          |
| aBTB (right) | A     | B   | C          |



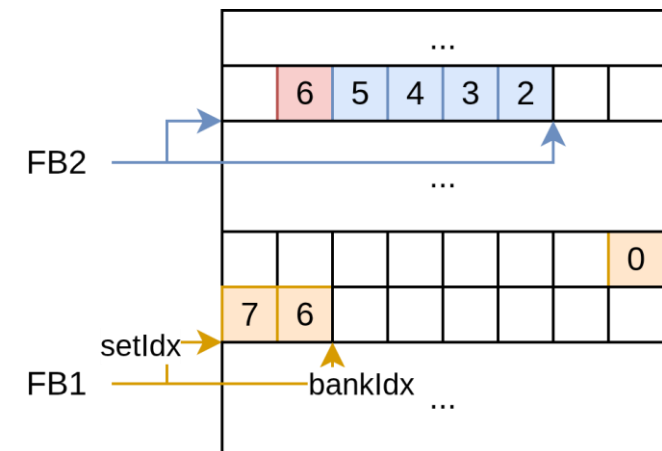
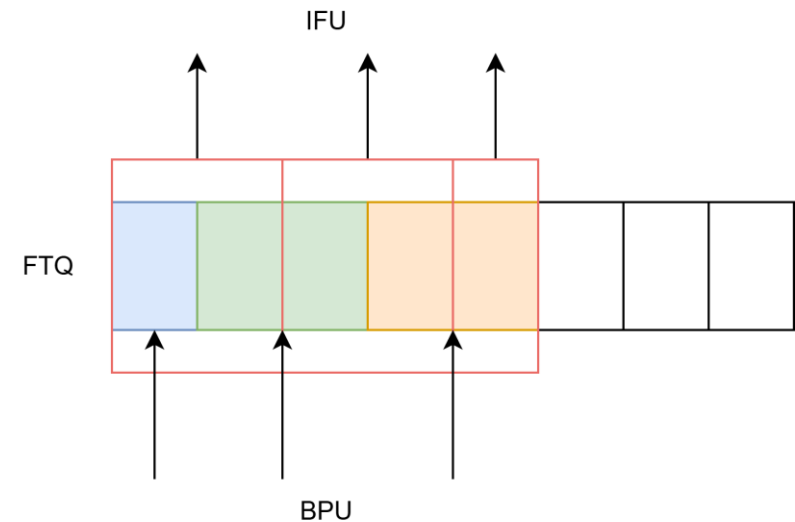


# S3 Predictors

- Main-BTB:
  - 4K entries with long target, 8K entries with short target
  - Half-aligned index
- TAGE-SC & ITTAGE & RAS:
  - Standard design with some tricks
  - Inherited from Kunminghu V2
  - Modified for BTB structure
  - Parameter optimization

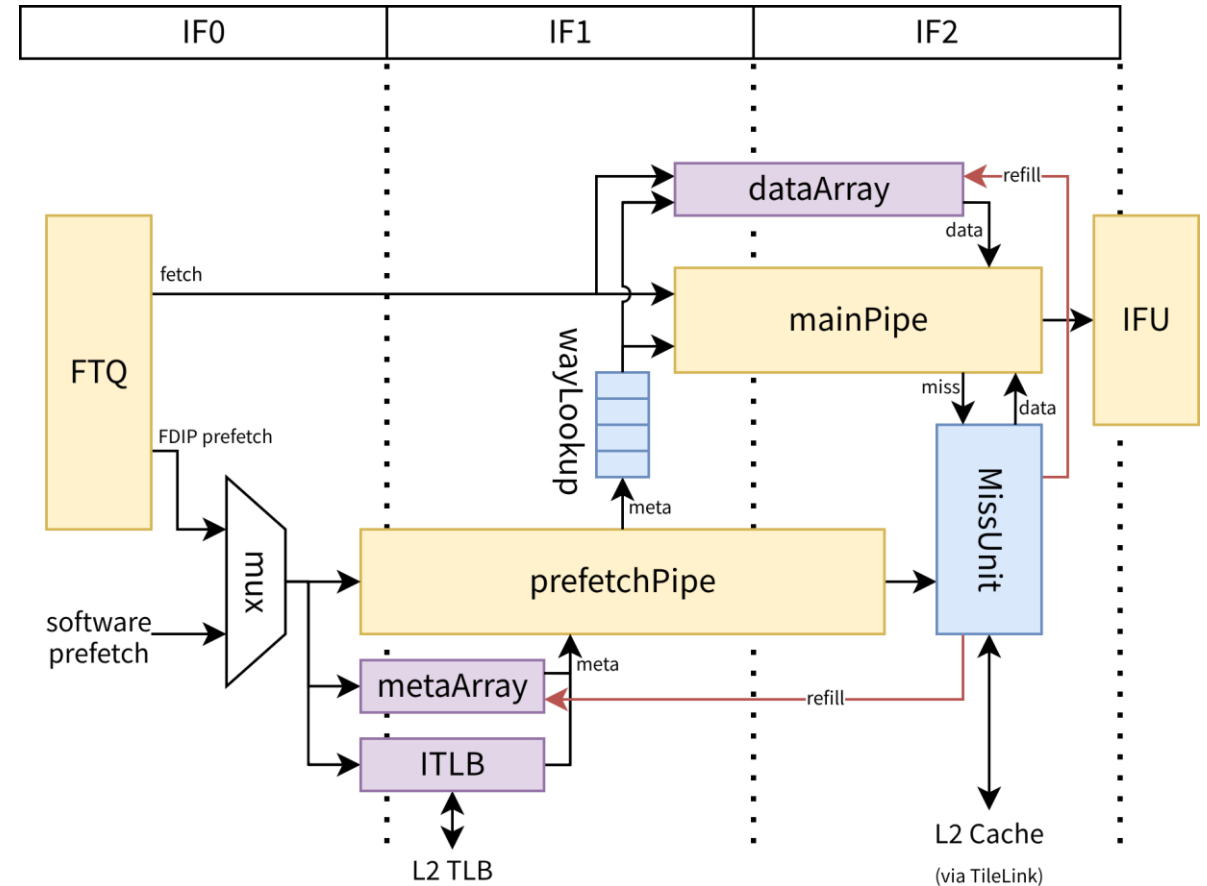


- Support for 2-taken and 2-fetch
  - Decoupled enqueue and dequeue
  - Flow control when dequeue
- Reduced redirect latency
- Train BPU when branches are resolved
  - Faster training usually means higher accuracy
  - Add a new resolve queue to cache branches
  - Complex control in resolve queue
- Manage queues according to life cycle



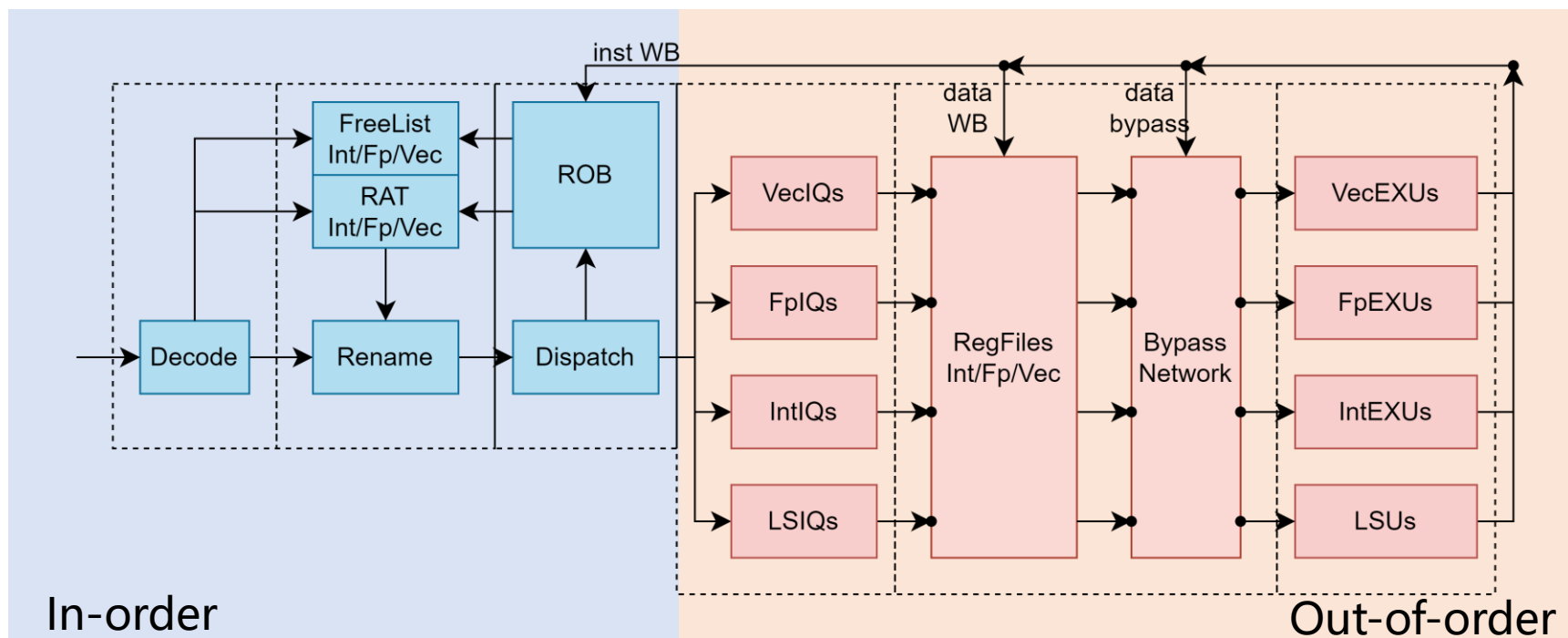
# ICache

- 64KB (4-way)
- 64B cacheline
- 8B bank fine-grained storage
- 4 + 10 MSHR
- Read hit banks only for low power
- Support Zicbop prefetch.i
- Support parity check
  - Support auto-recovery (re-fetch from L2 cache)





# Backend Architecture



# Backend Design Philosophy

- Primary goal: functionality, bandwidth and PPA trade-off
- 6-wide decode/rename/dispatch
- Snapshot, instruction fusion, move elimination and ROB compression
- Read regfile after issue
- Full support for RVA23 profile
  - Including **Vector** and **Hypervisor** extension



# Backend Related RVA23 Profiles

| Type          | Feature                                         |
|---------------|-------------------------------------------------|
| Unprivileged  | <b>V</b> , Zfh, Zvfh, Zimop, Zihintpause        |
| Exception     | Sstvala, Shtvala, Shvstvala, Smdbltrp, Ssdbltrp |
| Interrupt     | Smaia, Ssaia, Sstc, Smrnmi                      |
| Hypervisor    | <b>H</b> , Shgatpa, Shvstvecd                   |
| Perf counter  | Sscofpmf, Sscounterenw, Shcounterenw, Smcsrind  |
| State control | Smstateen, Ssstateen                            |
| Debug         | Sdtrig                                          |



# Collection of Features

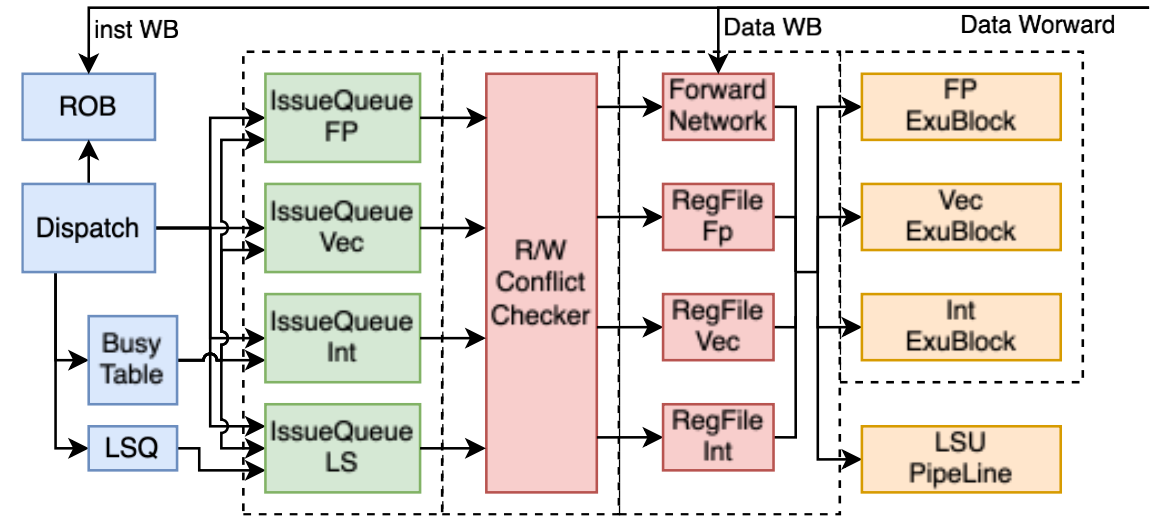
- Instruction fusion
  - Fuse adjacent uops to reduce dynamic instruction count
- Move elimination
  - Change the rename map directly
  - Mark as complete when move instruction enters ROB
- Rename snapshot
  - Re-rename from snapshot instead of committed instructions
  - Faster recovery after redirect
- Rob compression
  - Compress ROB entries according to certain rules





# Data Path

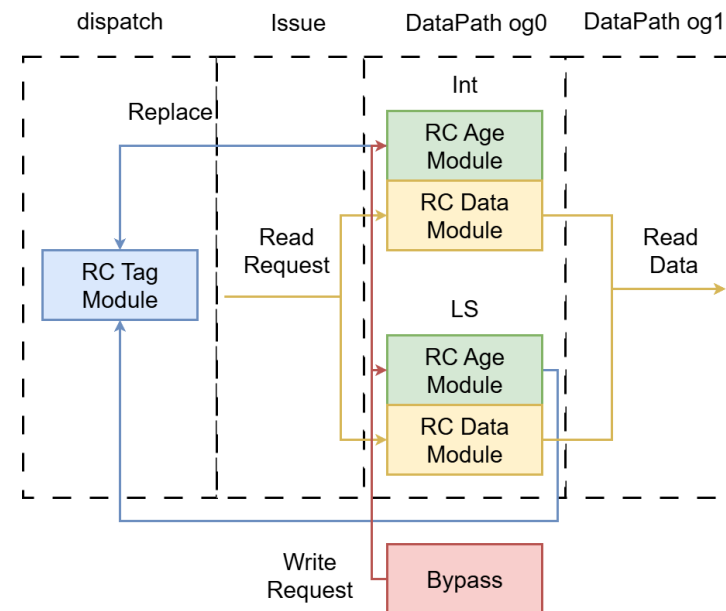
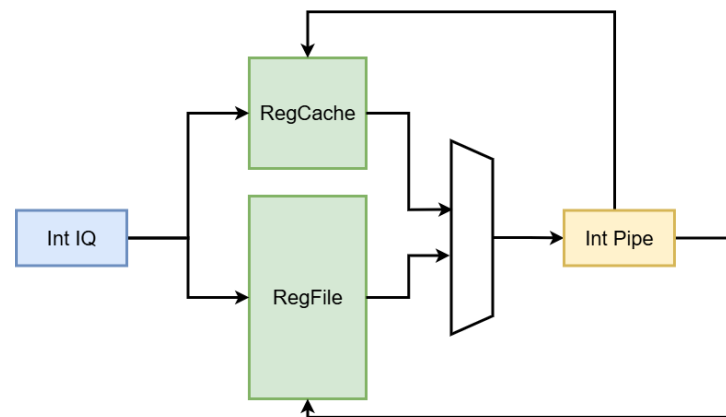
- Read regfile after issue
- Compared with otherwise:
  - Better timing and area
  - More function units and registers
  - More complex control path
- To mitigate:
  - Efficient arbitration of regfile read ports
  - Accurate speculative wakeup and cancelling





# Regfile Cache

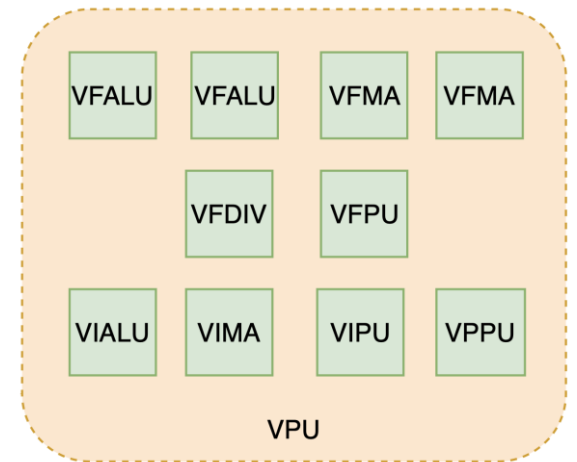
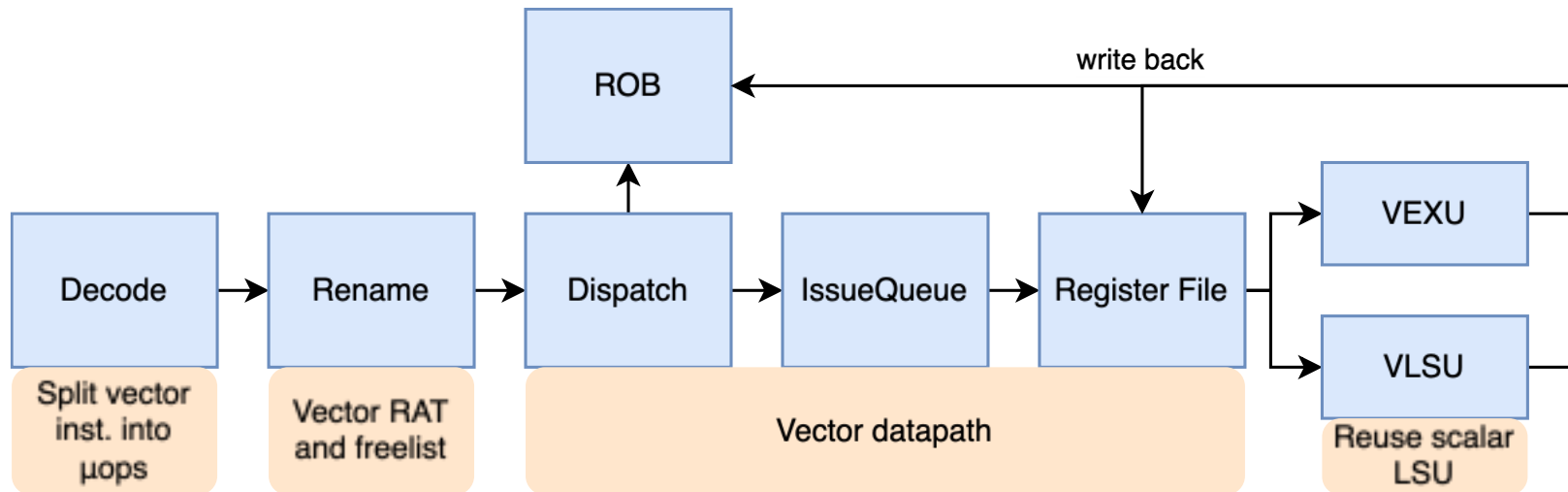
- Multiple functional units share a regfile read port
  - Conflicts on read ports
- Regfile is too big to add more read ports
- Add a small cache for regfile
  - Small means more read ports
- Reduce conflicts on original regfile
  - Same performance with fewer read ports
  - Fewer read port means better timing





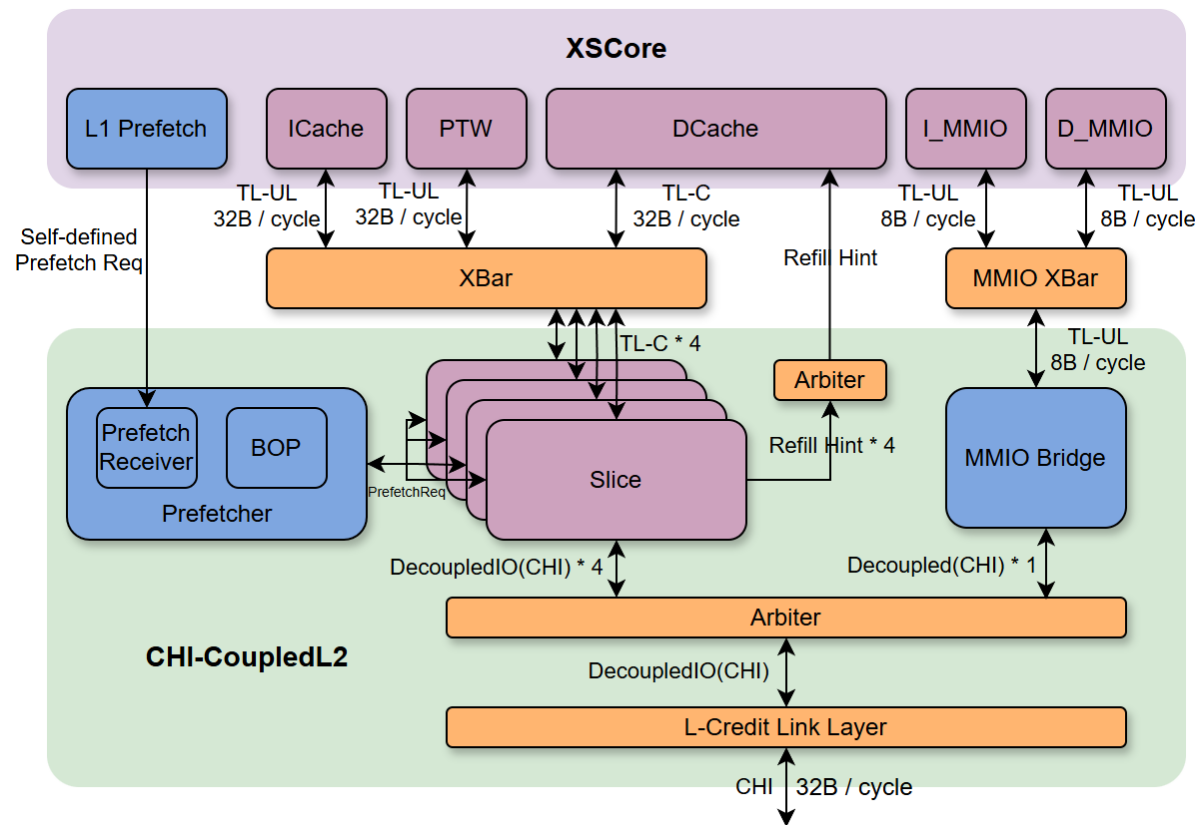
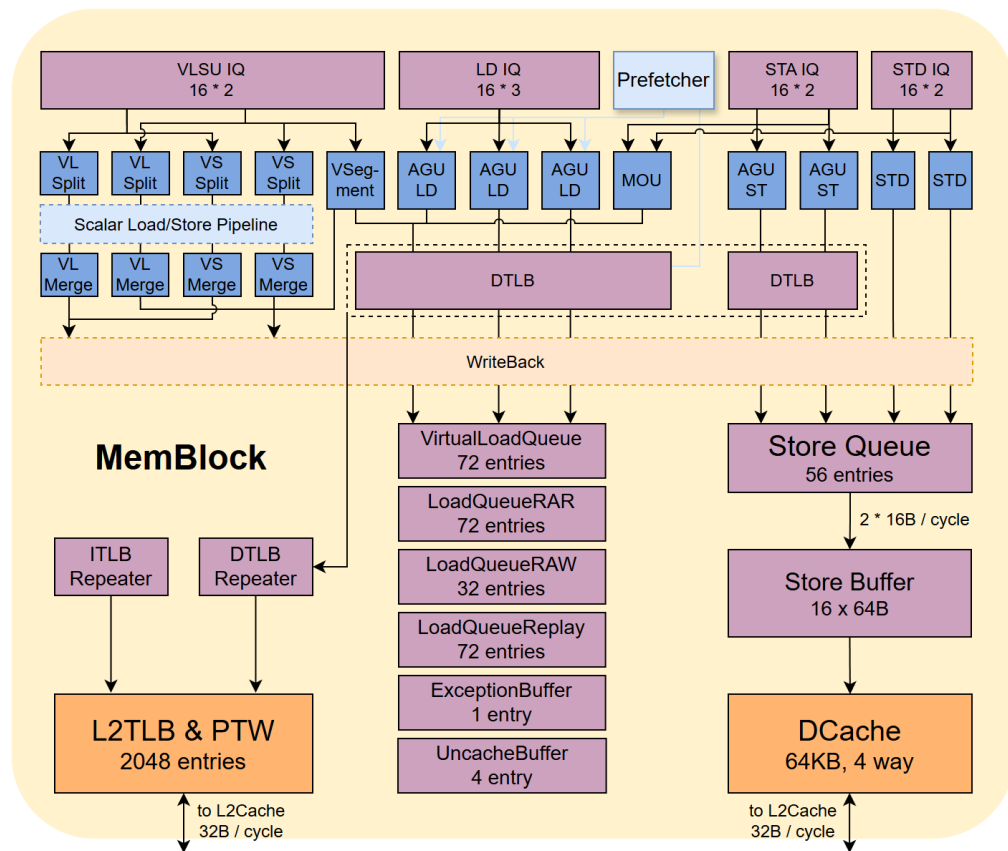
# V Extension

- Closely-coupled vector design, reuse existing pipeline
- Split vector instructions into uops at the granularity of vector registers
- VPU also process floating-point calculations





# Memblock and L2 Cache Architecture



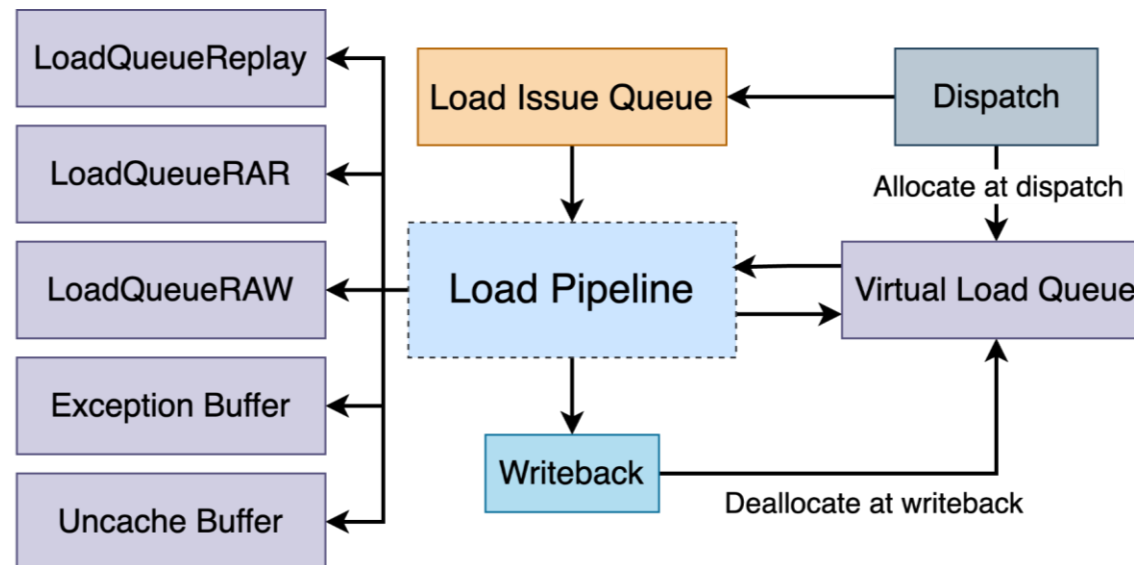


# Memblock and L2 Cache Design Philosophy

- Primary goals:
  - Functionality and safety
  - Latency, bandwidth and hit rate
- Support for V and H extension
- Support for CHI Issue E, C, B
- Distributed load queues
- Co-optimization between L1 and L2 Cache
- Multiple prefetch algorithms

# Distributed Load Queues

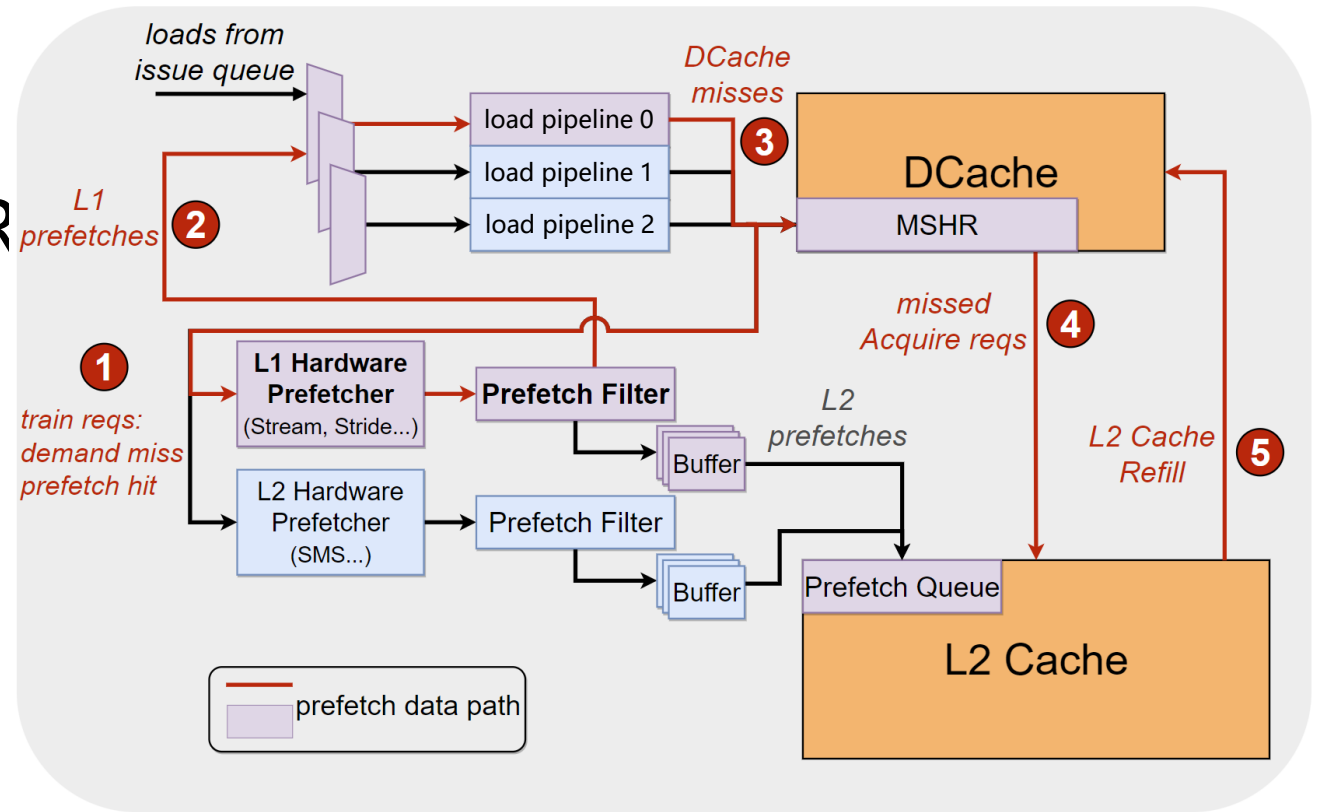
- Split load queue based on different responsibilities
- Simpler control, better timing and acceptable area
- Early commit: load instruction can be retired sequentially from load queue after write-back





# L1 DCache Prefetch

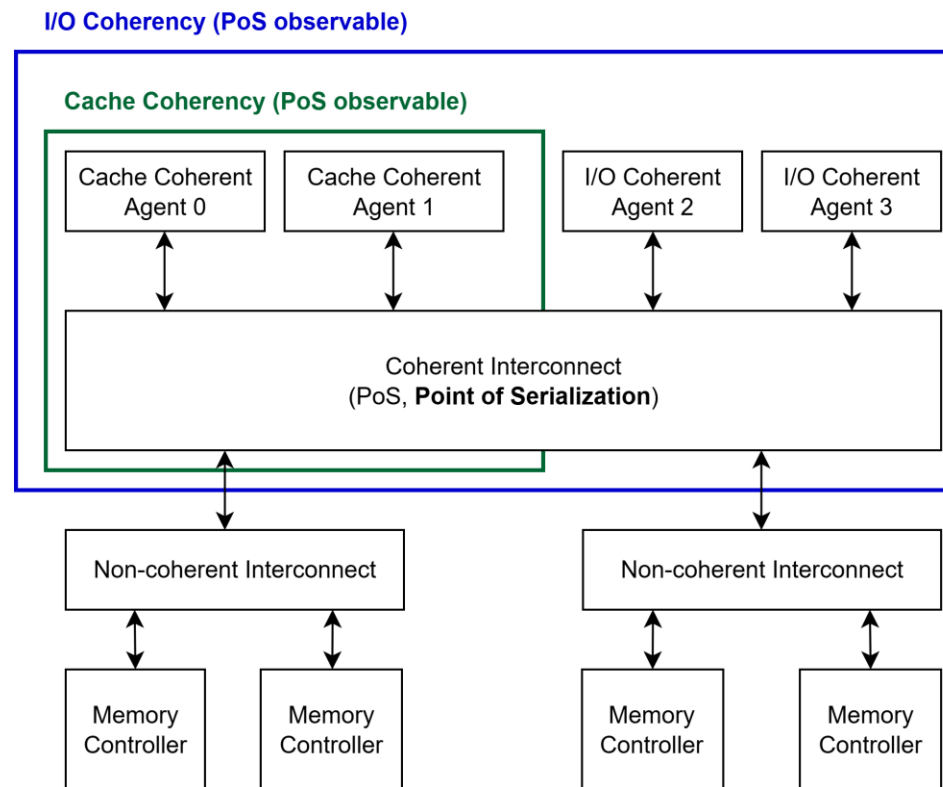
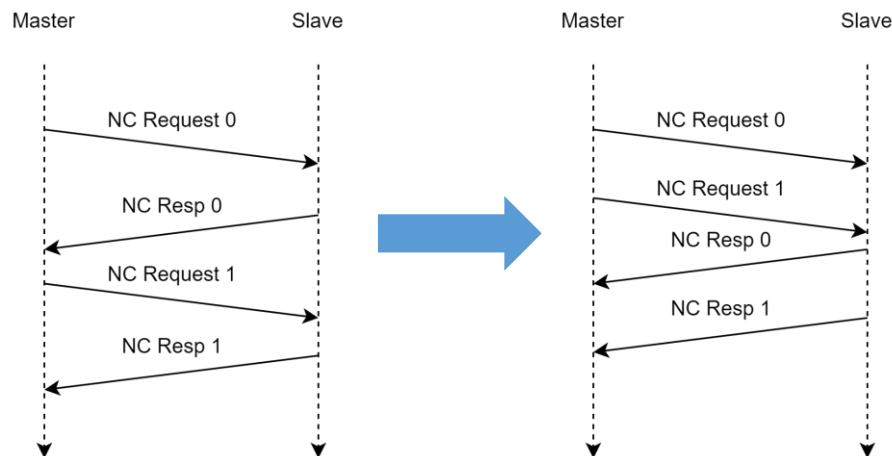
- Detect load pattern
  - Reuse load pipelines
  - Fetch data from D\$ MSHR
- 
- Stream:  $(x, x+1, x+2, \dots)$
  - Stride:  $(x, x+n, x+2n, \dots)$





# Outstanding IO Performance

- Svpbmt: memory attribute based on pages
  - NC: non-cacheable, RVWMO
  - IO: non-cacheable, sequential
- Out-of-order on NC area

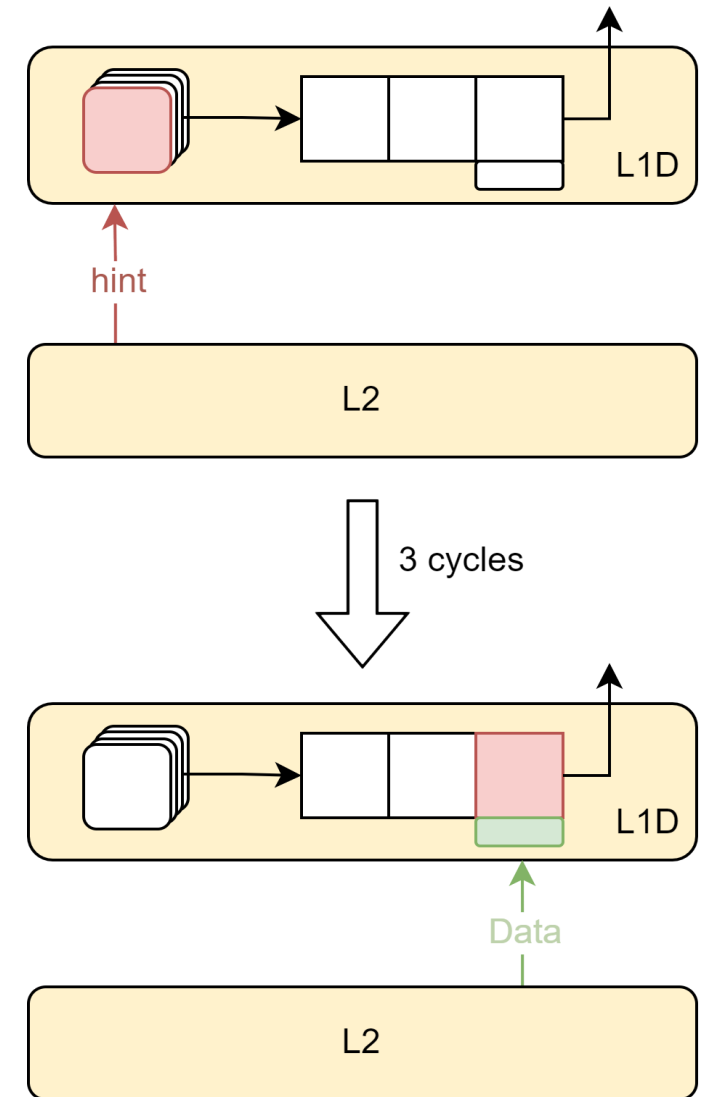






# L1-L2 Collaborative Optimization

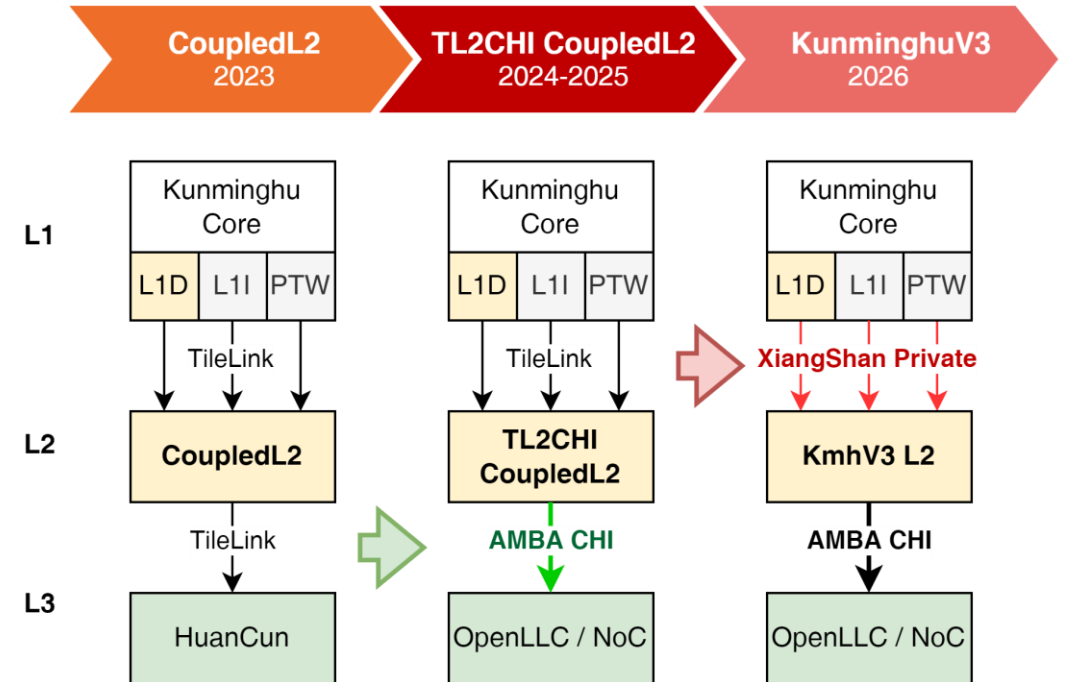
- 3 fixed cycles between load waking up and the use of refill data
- L2 Cache hints LSU to replay before refill
- Set up monitor to track info from main pipeline
- Calculate the exact hint timing
- Send hint signal to L1 3 cycles before refill





# Private Protocol between L1 and L2

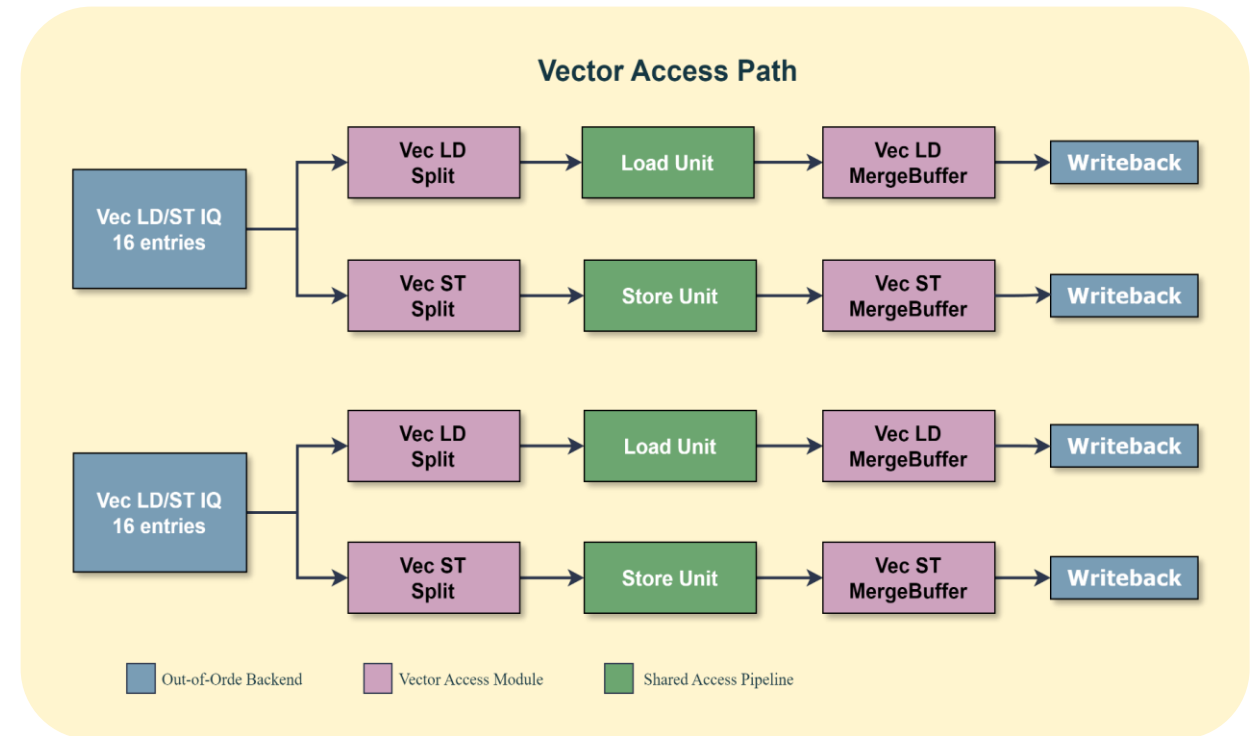
- Modifications have been made based on TileLink
  - To support CMO instructions
  - To co-optimize between L1 and L2
- Conversions between different coherence protocols are bad
  - More complicated verifications
  - Result in conservative design
- TileLink is functionally limited compared with CHI





# V Extension

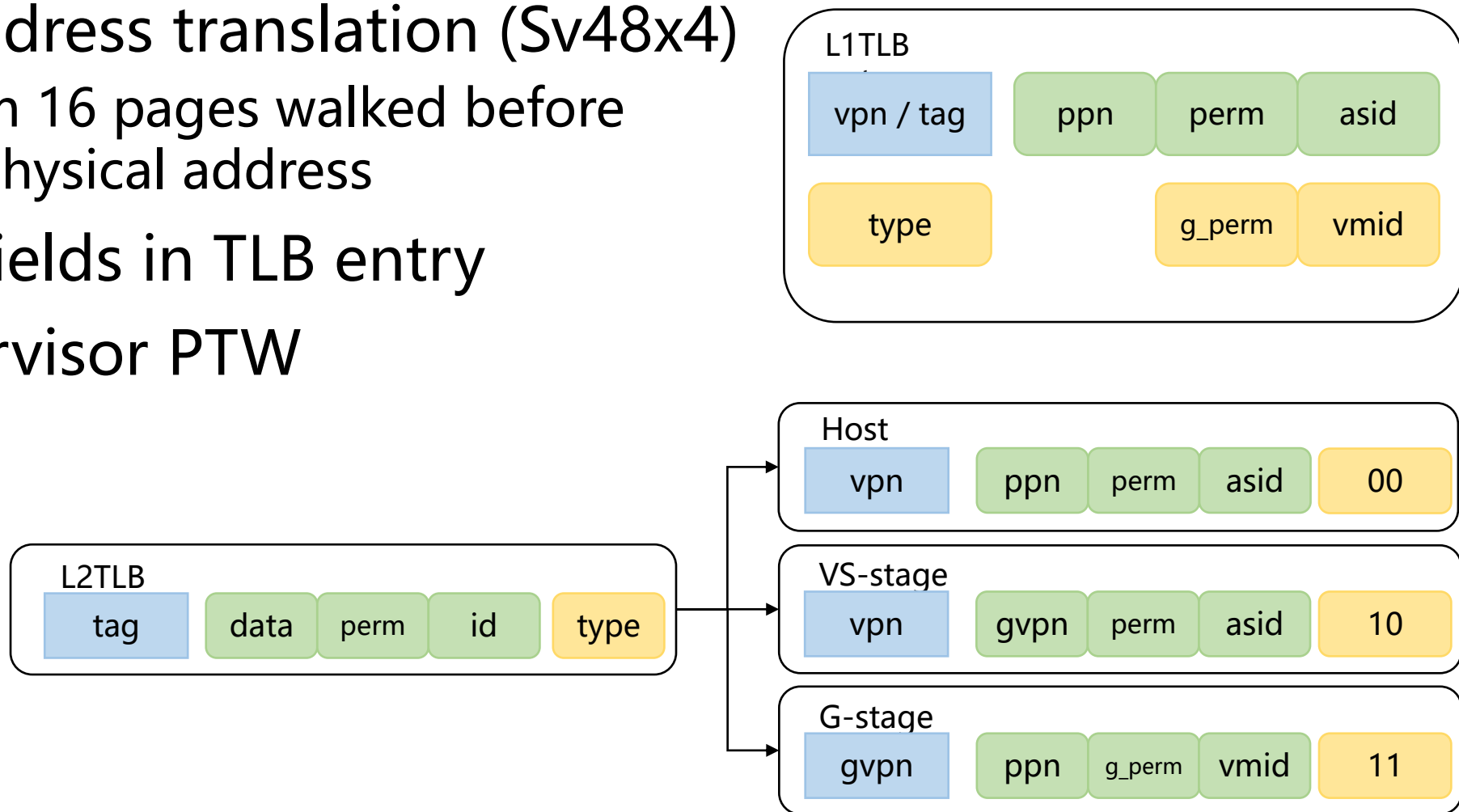
- Split vector instructions at the granularity of load/store operations
- Reuse scalar pipeline
- Bandwidth optimization for unit-stride access
  - One translation and cache request for contiguous memory access





# H Extension

- 2-stage address translation (Sv48x4)
  - Maximum 16 pages walked before getting physical address
- Add new fields in TLB entry
- New hypervisor PTW





# Conclusion

- High-bandwidth, decoupled frontend
  - High-accuracy BPU and high-bandwidth IFU
  - 2-taken and 2-fetch support in BPU and IFU
- Full-featured, high-bandwidth backend
  - Support for V extension
  - Many interesting techniques
- Secure and low-latency memory subsystem
  - Support for V and H extension
  - Co-optimization between L1 and L2 cache
- Questions are welcomed!